Functions

March 7, 2024

An Example

- Design a program to print lines containing a specific pattern.
- Break the task into three functions: getline, strindex, and main.
- getline: Reads a line into a string.
- strindex: Returns the position where one string begins in another.
- main: Orchestrates the process, finds lines with a specific pattern.

Pattern-searching Program

```
#include <stdio.h>
#define MAXLINE 1000

int getline(char line[], int max);
int strindex(char source[], char searchfor[]);
char pattern[] = "ould";

main() {
  char line[MAXLINE];
  int found = 0;
  while (getline(line, MAXLINE) > 0)    /* while a line exists       */
    if (strindex(line, pattern) >= 0) { /*   if the line has pattern */
      printf("%s", line);               /*       print line          */
      found++;
    }
  return found;
}
```

getline Function

```
/* getline: get line into s, return length */
int getline(char s[], int lim) {
  int c, i;
  i = 0;
  while (--lim > 0 && (c = getchar()) != EOF && c != '\n')
    s[i++] = c;
  if (c == '\n')
    s[i++] = c;
  s[i] = '\0';
  return i;
}
```

strindex Function

```
/* strindex: return index of t in s, -1 if none */
int strindex(char s[], char t[]) {
  int i, j, k;
  for (i = 0; s[i] != '\0'; i++) {
    for (j = i, k = 0; t[k] != '\0' && s[j] == t[k]; j++, k++)
        ;
    if (k > 0 && t[k] == '\0')
      return i;
  }
  return -1;
}
```

Local Variables in Functions

- Variables like `line`, `found` in `main` are local.
- Local variables are private to functions.
- They come into existence when function is called and disappear when function is exited.
- Known as automatic variables.
- Automatic variables don't retain values between calls.
- They must be explicitly set upon each entry.
- Failure to set them may result in garbage values.

Compiling and Loading C Programs

- C programs may reside in one or more source files.
- Source files are human-readable files that contain the program's source code; these have .c extensions.
- Source files can be compiled separately and loaded together.
- Details vary from system to system.
- Example compilation on Ubuntu: `gcc main.c getline.c strindex.c`

Object files and Executable files

- Compilation involves translating the source code into machine code specific to the target architecture. This machine code is then stored in object files. They are not directly executable.
- Object files have extensions like .o (for Unix-like systems), .obj (for Windows), or .out.
- Executable files contain machine code that the computer's hardware can execute directly. These are generated by linking one or more object files together, along with any necessary libraries.
- On Unix-like systems, the executable file might have no extension or end with .out, while on Windows, it usually ends with .exe.
- When you run an executable file, it starts the program and executes the instructions written in the source code.

Functions Returning Non-integers

- Functions in C can return types other than `int`.
- Numeric functions like `sqrt`, `sin`, and `cos` return `double`.
- Illustration: Write and use `atof(s)` to convert a string to `double`.
- `atof` is an extension of `atoi` with enhanced functionality.

atof Function

```
#include <ctype.h>
/* atof: convert string s to double */
double atof(char s[]) {
  double val, power;
  int i, sign;

  for (i = 0; isspace(s[i]); i++)
      ;
  sign = (s[i] == '-') ? -1 : 1;
  if (s[i] == '+' || s[i] == '-')
      i++;
  for (val = 0.0; isdigit(s[i]); i++)
      val = 10.0 * val + (s[i] - '0');
  if (s[i] == '.')
      i++;
  for (power = 1.0; isdigit(s[i]); i++) {
    val = 10.0 * val + (s[i] - '0');
    power *= 10;
  }
  return sign * val / power;
}
```

main Function

```
#include <stdio.h>

#define MAXLINE 100

/* rudimentary calculator */
main() {
  double sum, atof(char []);
  char line[MAXLINE];
  int getline(char line[], int max);

  sum = 0;
  while (getline(line, MAXLINE) > 0)
    printf("\t%g\n", sum += atof(line));
  return 0;
}
```

Type Declaration for `atof`

```
double sum, atof(char []);
```

- `sum` is a `double` variable.
- `atof` is a function that takes one `char[]` argument and returns a `double`.

Type Consistency

- Be consistent between declaration and definition. Error otherwise.
- It's not a good idea to skip declarations.
- Implicit declarations lead to potential errors.
- Say, f is undeclared, and "f(" occurs in an expression
- Then, "f" is a function that returns int; nothing is assumed about arguments.
- If a function declaration has no arguments, nothing is to be assumed about them; all parameter checking is turned off.

atoi Function

```
/* atoi: convert string s to integer using atof */
int atoi(char s[]) {
  double atof(char s[]);
  return (int) atof(s);
}
```

Casting can be used to explicitly convert types.

External Variables

- alternative to automatic variables.
- defined outside of any function.
- accessible globally by any function.
- can be used for data communication between functions.
- persist beyond function calls, retaining values.
- provide an alternative to function arguments for data communication.

External Variables

- External linkage: all references to external objects by the same name, even from functions compiled separately, are references to the same thing.
- Greater scope and lifetime make external variables useful.
- Over-reliance on them can lead to programs with too many data connections between functions, unexpected changes, and difficulties in modification.

Defining of External Variables

- Define exactly once outside any function.
- Declare in each function that accesses it.
- Declaration may be `extern` or implicit from context.
- A global variable definition is implicitly available inside every function below it in its file.
- We now rewrite an above program with max, line, longest as global variables.

External Variables and Scope

```
#include <stdio.h>
#define MAXLINE 1000

int max; char line[MAXLINE]; char longest[MAXLINE];

int getline(void); void copy(void);

int main(){
    int len;
    max = 0;

    while ((len = getline()) > 0){
        if (len > max){
            max = len; copy();
        }
    }
    if (max > 0){
        printf("%s", longest);
    }
    return 0;
}
```

External Variables and Scope

```
int getline(void){
    int c, i;
    for (i = 0; i<MAXLINE-1 && (c=getchar())!=EOF && c!='\n'; ++i)
        line[i] = c;

    if (c == '\n'){
        line[i] = c;
        ++i;
    }

    line[i] = '\0';
    return i;
}

void copy(void){
    int i;
    i = 0;

    while ((longest[i] = line[i]) != '\0')
        ++i;
}
```

External variables

- External variables' definitions are often placed at the beginning of the source file.
- `extern` declarations can be omitted if the definition precedes use in the source file.
- Common practice is to omit `extern` declarations in one-file-programs.
- Header files collect declarations, included by `#include` in source files.
- Prototypes for functions without arguments should use `void` explicitly.